

Abstract

This document provides a comprehensive technical reference and architectural guide to Web Authentication (WebAuthn), the W3C and FIDO Alliance standard establishing passwordless authentication across the web ecosystem. Traditional shared-secret mechanisms, such as passwords and legacy multi-factor authentication (MFA), suffer from inherent vulnerabilities, including phishing, credential stuffing, and server-side database breaches. WebAuthn eliminates shared secrets by leveraging asymmetric public-key cryptography, where private keys remain strictly isolated within hardware-backed security modules while public keys are registered with Relying Parties (RPs).

The document details the foundational security properties of WebAuthn, being **Strong** (hardware-isolated), **Scoped** (cryptographically origin-bound), and **Attested** (verifiable authenticator provenance). It clarifies the industry terminology surrounding **Passkeys**, distinguishing between consumer-centric, cloud-synced discoverable credentials (e.g., Apple iCloud Keychain, Google Password Manager) and enterprise-bound, single-device hardware credentials (e.g., FIPS-certified YubiKeys, TPMs).

Through step-by-step sequence flows and diagrams, the document orchestrates the core protocol **ceremonies**, specifically **Registration** and **Authentication**, mapping the precise interaction between Relying Parties, User Agents, Users, and authenticators.

From a threat model perspective, the document analyzes how cryptographic origin binding (**clientDataJSON**) inherently neutralizes Man-in-the-Middle (MitM) and phishing attacks, while evaluating proximity-verified cross-device authentication. Finally, a technical deep dive provides implementation specifications for Relying Party developers, including browser JavaScript APIs, attestation statement format parsing (Packed, TPM, Apple Anonymous), FIDO Metadata Service (MDS3) integration for vendor AAGUID validation, and advanced protocol extensions (**prf** for client-side encryption, **largeBlob**, and **credProps**).

Document Index

1. Abstract
2. Terminology
3. Reasons for its design and standardization
4. What is WebAuthN?
5. Passkey branding
6. Overview
 1. WebAuthN Roles
 2. Registration Ceremony
 3. Authentication Ceremony
 4. Discoverable vs Non Discoverable Credentials
7. Security Considerations
 1. Cryptographic Origin Binding
 2. Cross-Device Roaming
8. Support
9. Deep Dive
 1. WebAuthn API

2. Credential Types
 1. Discoverable Credentials (formerly "Resident Keys")
 2. Non-Discoverable Credentials (or "Server-Side" Credentials)
 3. Authenticators Attestation
 1. Why do we need attestation?
 2. Attestation Formats
 3. FIDO Metadata Service
 4. Use case: Validate that an authenticator Vendor/Model
 4. WebAuthn Extensions
 10. Conclusion
 11. Author Bios
 12. Change Log
 13. References (foot notes)
-

Document's section descriptions

1. Abstract

- **Purpose:** To provide a high-level overview of this document.

2. Terminology

- **Purpose:** Defines specific terms and concepts used throughout the document.
- **Summary:** Provides precise definitions for foundational terms such as "authenticator", "relying party", "public key credential", "attestation", "user verification", and "ceremony".

3. Reasons for its design and standardization

- **Purpose:** To explain the fundamental security problems WebAuthn aims to solve and why a standardized approach was necessary.
- **Summary:** Passwords are inherently insecure and vulnerable to phishing, data breaches, and credential stuffing. This section outlines how WebAuthn addresses these vulnerabilities by replacing shared secrets (passwords) with asymmetric public-key cryptography.

4. What is WebAuthN?

- **Purpose:** To introduce Web Authentication (WebAuthn) and public key cryptography as the modern solution to password vulnerabilities.
- **Summary:** WebAuthn is an API specification by W3C and FIDO that replaces passwords with public key cryptography. Instead of sharing a secret, a private key is stored securely on the user's device while the public key is sent to the server. The section outlines WebAuthn's three major security properties: it is **Strong** (often backed by hardware security modules), **Scoped** (origin-bound to prevent phishing), and **Attested** (servers can verify the public key comes from a trusted authenticator).

5. Passkey branding

- **Purpose:** To clarify the terminology shift and consumer-facing branding of WebAuthn credentials.
- **Summary:** This section explains the term "passkey," which was introduced by an industry coalition (including Apple, Google, and Microsoft) to serve as a more user-friendly, consumer-facing term for **discoverable** FIDO2/WebAuthn credentials. Passkeys can often be synced across a user's devices via cloud ecosystems (like iCloud Keychain or Google Password Manager), making passwordless login much more convenient. Clarifies that not all

WebAuthn credentials are passkeys, categorizing credentials into Discoverable Credentials (Passkeys) and Non-Discoverable Credentials based on where credential data is stored.

6. Overview

- **Purpose:** To provide a high-level technical explanation of how the WebAuthn protocol operates across components.
- **Summary:** This section serves as an umbrella for the core mechanics of WebAuthn. It explains the relationship between the Relying Party (the website), the Client (the web browser), and the Authenticator (the hardware key or biometric sensor).

1. WebAuthN Roles

- **Purpose:** To define the primary entities and terms involved in the WebAuthn ecosystem.
- **Summary:** Outlines four key roles: the **Relying Party (RP)**, the **WebAuthn Client Device / User Agent**, the **User**, and the **Authenticator** (platform authenticator like biometrics or roaming authenticator like YubiKey).

2. Registration Ceremony

- **Purpose:** To detail the technical flow and data exchange during credential registration.
- **Summary:** Explains the process when a user creates an account or adds an authenticator. The Relying Party issues a challenge, the authenticator generates a new public-private key pair, stores the private key securely, and returns the public key and attestation statement to the server. [Include DIAGRAM]

3. Authentication Ceremony

- **Purpose:** To detail the ceremony and verification steps during user login.
- **Summary:** Explains the login flow. The Relying Party sends a challenge to the client, the authenticator requests user verification/presence, signs the challenge with the stored private key, and returns the signature to the RP for verification using the public key. [Include DIAGRAM]

4. Discoverable vs Non Discoverable Credentials

- **Purpose:** Describe the types of credentials available and ceremony differences for each.
- **Summary:** Compares how discoverable credentials enable username-less authentication flows by storing credential IDs on the authenticator versus non-discoverable credentials which require the RP server to provide the credential ID during the challenge.

7. Security Considerations

- **Purpose:** Analyzes the security model, threat vector protections, and remaining security risks related to WebAuthn.
- **Summary:** Discusses how WebAuthn protects against phishing, Man-in-the-Middle (MitM), replay attacks, and credential stuffing, alongside considerations for physical device theft and key management.

1. Cryptographic Origin Binding

- **Purpose:** To explain how WebAuthn ties credentials to domain origins to prevent phishing.
- **Summary:** Details how the browser injects the exact origin (scheme, domain, port) into the client data hash signed by the authenticator, ensuring credentials created for **evil.com** cannot be used on **bank.com**.

TODO: Check current spec, it seems it uses RP ID which doesn't have scheme/port.

2. Cross-Device Roaming

- **Purpose:** To evaluate the security and threat vectors of cross-device authentication mechanisms (FIDO Cross-Device Authentication / caBLE / Hybrid).
- **Summary:** Examines how QR-code based hybrid transport enables mobile devices to act as authenticators for desktop browsers over Bluetooth low energy (BLE), maintaining proximity verification while expanding usability.

8. Support

- **Purpose:** To outline the adoption, compatibility, and readiness of WebAuthn across platforms.
- **Summary:** Details implementation support across major web browsers (Chrome, Firefox, Safari, Edge), operating systems (Windows, macOS, Linux, Android, iOS), and security key manufacturers (Yubico, Feitian, SoloKeys).

9. Deep Dive

- **Purpose:** To explore technical API specifications, attestation mechanics, and protocol extensions in depth.
- **Summary:** Provides granular developer and security engineering reference material on browser APIs, credential storage modes, attestation validation, and advanced protocol extensions.

1. WebAuthn API

- **Purpose:** To introduce the technical usage and implementation of the Web Authentication JavaScript API.
- **Summary:** Explains core browser methods (`navigator.credentials.create()` and `navigator.credentials.get()`), detailing dictionary options, binary buffer conversions, and script execution without exposing private keys.

2. Credential Types

- **Purpose:** To define the architectural differences and storage implementations of WebAuthn credentials.
- **Summary:** Deep dives into storage limitations, key wrapping strategies, and client vs server storage models for discoverable and non-discoverable credentials.

1. Discoverable Credentials (formerly "Resident Keys")

- **Purpose:** To detail the mechanics, storage overhead, and UX implications of resident/discoverable keys.
- **Summary:** Explains how credentials storing both private key and user handle inside authenticator storage enable true 1-step passwordless login, along with authenticator capacity limits.

2. Non-Discoverable Credentials (or "Server-Side" Credentials)

- **Purpose:** To detail traditional FIDO credentials where key handles are stored by the Relying Party.
- **Summary:** Covers how authenticators encrypt private keys into credential IDs sent to RPs, enabling unlimited credential storage without filling up physical security key memory.

3. Authenticators Attestation

- **Purpose:** To detail how Relying Parties verify authenticators' identity, cryptographic provenance, and security compliance.

- **Summary:** Covers attestation statements, root certificates, model verification, and trust chains required for compliance-bound enterprise environments.

1. Why do we need attestation?

- **Purpose:** To justify the need for cryptographic proof of authenticator hardware model and vendor.
- **Summary:** Explains how attestation allows enterprise Relying Parties to enforce security policies (e.g., requiring FIPS 140-2 Level 3 certified authenticators) before accepting registrations.

2. Attestation Formats

- **Purpose:** To detail standardized attestation data formats defined in the WebAuthn spec.
- **Summary:** Analyzes formats including Packed, TPM (Trusted Platform Module), Android Key/SafetyNet/Play Integrity, Apple Anonymous Attestation, and FIDO U2F.

3. FIDO Metadata Service

- **Purpose:** To explain how RPs consume the FIDO Metadata Service (MDS) to evaluate authenticators.
- **Summary:** Outlines the architecture of FIDO MDS3, payload downloading, JSON Web Signatures (JWS) verification, and extracting authenticator metadata statements and certification statuses.

4. Use case: Validate that an authenticator Vendor/Model

- **Purpose:** To provide an end-to-end walkthrough of validating authenticators against an enterprise whitelist.
- **Summary:** Step-by-step logic showing how an RP parses AAGUIDs (Authenticator Attestation GUIDs), verifies attestation signature chains against MDS roots, and enforces access control decisions.

4. WebAuthn Extensions

- **Purpose:** Describes the mechanism for extending the WebAuthn API to suit specialized cryptographic and protocol use cases.
- **Summary:** Lists and explains standardized extensions such as **appid** (FIDO U2F backward compatibility), **credProps** (discoverability confirmation), **prf** (pseudo-random function for encryption key derivation), and **largeBlob** (storing arbitrary encrypted payloads).